



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO
DIGITAL
CARRERA DE SOFTWARE

TEMA:

Práctica en clase

GRUPO:

CAJAS IBARRA IRVIN MARCELO
PANAMA MURILLO MOISES ANTONIO
TAIPE MORA ZAIDA MELISSA

DOCENTE:

Guerrero Ulloa Gleiston Ciceron

MATERIA:

Aplicaciones Web – A – [5to Nivel]

Quevedo – Ecuador 2026

INTRODUCCIÓN

El desarrollo de aplicaciones web del lado del servidor constituye una parte fundamental dentro de la ingeniería de software, ya que permite crear sistemas dinámicos capaces de procesar información, gestionar formularios, controlar sesiones y responder a las solicitudes realizadas por los usuarios desde un navegador web. En este contexto, Java se presenta como una tecnología ampliamente utilizada para el desarrollo de aplicaciones empresariales debido a su robustez, portabilidad y compatibilidad con servidores de aplicaciones como Apache Tomcat.

La presente práctica formativa tiene como finalidad implementar un proyecto web utilizando Java, JSP y Servlets, aplicando los conceptos correspondientes a la Unidad II de la asignatura Aplicaciones Web. A través de esta actividad se busca comprender el funcionamiento de una aplicación web Java basada en el patrón petición-respuesta, donde el navegador envía datos mediante formularios y el servidor los procesa mediante clases Servlet.

Durante el desarrollo de la práctica se trabajará con una estructura de proyecto Maven WebApp, configurando las dependencias necesarias para Jakarta Servlet y JSP. Además, se implementará un formulario de registro de usuario, una vista JSP para la interfaz y un Servlet encargado de procesar la información recibida. Este proceso permitirá evidenciar el ciclo de vida de un Servlet, el uso de métodos como doGet y doPost, la validación de datos del lado del servidor y la comunicación entre la lógica backend y la vista.

Finalmente, el informe incluirá el código implementado y capturas del paso a paso, así como evidencias de los resultados obtenidos durante la ejecución del proyecto. De esta manera, la práctica contribuye al fortalecimiento de competencias relacionadas con el desarrollo web dinámico, la programación del lado del servidor y el uso de herramientas modernas dentro del ecosistema Java.

OBJETIVO

Desarrollar una aplicación web dinámica utilizando JSP y Servlets en Java, implementando un formulario de registro de usuarios con validación de datos del lado del servidor. Además, aplicar el uso de Apache Tomcat como servidor web, Maven para la gestión de dependencias y la arquitectura cliente-servidor para comprender el funcionamiento de aplicaciones web empresariales desarrolladas con tecnologías Java.

HERRAMIENTAS UTILIZADAS

- IntelliJ IDEA Community Edition.
- Java Development Kit (JDK) 21.
- Apache Maven.
- Apache Tomcat 10.1.
- Jakarta Servlet API 6.0.
- JSP (Java Server Pages).
- JSTL (Jakarta Standard Tag Library).
- Google Chrome.
- Sistema Operativo Windows 11.

MARCO TEÓRICO Y DESARROLLO

De Java EE a Jakarta EE

Java EE (Java Enterprise Edition) fue el conjunto de especificaciones para aplicaciones empresariales Java gestionado por Oracle. En el año 2017, Oracle donó el proyecto a la Eclipse Foundation, que lo renombró como Jakarta EE a partir de la versión 9. El cambio más visible de esta transición es el prefijo de los paquetes: `javax.servlet.*` (Java EE ≤ 8) se convierte en `jakarta.servlet.*` (Jakarta EE ≥ 9). Apache Tomcat 10.x implementa Jakarta EE 9+, por lo que en esta práctica se utiliza Apache Tomcat 10.1.x con Jakarta Servlet 6.0 (`jakarta.servlet.*`) y Java 21 LTS.

Arquitectura de una aplicación web Java (WAR)

Una aplicación web Java tradicional se empaqueta como un archivo WAR (Web Application Archive), el cual contiene una estructura organizada en distintos directorios. En la raíz se ubican las páginas JSP, archivos HTML, CSS, JS e imágenes accesibles directamente por el cliente. La carpeta `WEB-INF/` contiene el descriptor de despliegue `web.xml`, encargado del mapeo de Servlets, filtros y parámetros; la subcarpeta `WEB-INF/classes/` almacena las clases Java compiladas (Servlets, beans, etc.); y `WEB-INF/lib/` contiene las librerías

JAR necesarias, como JSTL, drivers JDBC o BCrypt. Finalmente, META-INF/ almacena los metadatos del archivo WAR.

Todo lo que se encuentra dentro de WEB-INF/ no es accesible directamente por el navegador. Por esta razón, los archivos JSP que contienen lógica de negocio y que no deben ser accedidos directamente se colocan en WEB-INF/views/, de modo que únicamente el Servlet pueda realizar un *forward* hacia ellos.

Ciclo de vida de un Servlet

Un Servlet es una clase Java que extiende de HttpServlet y se encarga de procesar las solicitudes HTTP recibidas por el servidor. Es el equivalente, dentro del ecosistema Java, a un script PHP o a un controlador en ASP.NET MVC. Apache Tomcat actúa como contenedor de Servlets: se encarga de cargar las clases, gestionar su ciclo de vida y entregarles las solicitudes HTTP correspondientes.

El ciclo de vida de un Servlet está compuesto por los siguientes métodos:

- **init()**: se ejecuta una sola vez al iniciar el Servlet. Se utiliza para inicializar recursos costosos, como pools de conexión a bases de datos, configuraciones o cachés. Cumple una función similar a la de un constructor.
- **service()**: se ejecuta en cada solicitud recibida y delega el procesamiento al método correspondiente según el verbo HTTP utilizado (GET, POST, PUT, DELETE). Raramente se sobrescribe de forma directa.
- **doGet()**: se ejecuta en cada solicitud GET. Se utiliza para procesar solicitudes de lectura, como mostrar formularios, listar datos o navegar entre vistas.
- **doPost()**: se ejecuta en cada solicitud POST. Se utiliza para procesar el envío de formularios, como registrar usuarios, autenticar o crear nuevos recursos.
- **destroy()**: se ejecuta una sola vez al finalizar el Servlet, y se utiliza para liberar recursos, como cerrar conexiones abiertas.

Conceptos clave del Servlet implementado

Para la implementación del Servlet de registro de usuarios se emplearon los siguientes conceptos fundamentales:

- **@WebServlet**: anotación que mapea una URL específica (por ejemplo, /registro) a una clase Servlet. Equivale a la configuración tradicional realizada en el archivo web.xml, pero de forma más concisa.
- **forward()**: método mediante el cual el Servlet cede el control de la solicitud a una página JSP, sin que la URL del navegador cambie. Resulta útil para mostrar errores de validación, ya que se preserva el estado de la solicitud original.
- **sendRedirect()**: método que provoca que el navegador realice una nueva solicitud GET hacia la URL indicada, cambiando la URL visible. Su uso es propio del patrón Post/Redirect/Get (PRG), que evita el reenvío accidental de formularios.
- **setAttribute()**: permite colocar datos dentro del objeto request para que posteriormente sean leídos por la vista JSP mediante Expression Language, por ejemplo `${nombre}`.
- **BCrypt.hashpw()**: método utilizado para generar el hash de la contraseña del usuario, aplicando un factor de costo configurable (un costo de 12 equivale a 4096 iteraciones), lo que dificulta los ataques de fuerza bruta.

JSP: vistas dinámicas con Expression Language y JSTL

Una página JSP es un documento que combina HTML con elementos dinámicos de Java. Cuando es solicitada por primera vez, Apache Tomcat la compila automáticamente a un Servlet y la almacena en caché; las solicitudes posteriores se procesan directamente sobre el Servlet ya compilado, lo cual mejora el rendimiento.

Entre los elementos JSP más utilizados se encuentran la directiva page (`<%@ page ... %>`), empleada para configurar la codificación, las importaciones y el tipo de contenido; la directiva taglib (`<%@ taglib ... %>`), utilizada para importar librerías JSTL o etiquetas personalizadas; y los *scriptlets* (`<% código Java %>`), que permiten incrustar código Java directamente en la vista, aunque su uso se considera una mala práctica.

Para la presentación de datos se emplea Expression Language (EL), mediante la sintaxis `${variable}`, que permite mostrar valores de atributos del request o de la sesión. Adicionalmente, se utilizan las etiquetas de JSTL: `<c:out value="${x}"/>`, que muestra y escapa automáticamente el HTML como medida anti-XSS; `<c:if test="${cond}">`, utilizada para evaluar condicionales; y `<c:forEach items="${lista}">`, empleada para iterar sobre colecciones de datos.

La regla de oro en el desarrollo de JSP establece que no deben utilizarse *scriptlets*, ya que mezclan la lógica de negocio con la presentación, dificultando las pruebas y el mantenimiento del código. La práctica moderna consiste en utilizar Expression Language junto con JSTL para la presentación, dejando toda la lógica de negocio en el Servlet. Un JSP correctamente escrito no debería contener ninguna línea de código Java embebido.

Cabeceras de seguridad HTTP

Como parte de las buenas prácticas de seguridad aplicadas al proyecto, se consideraron las siguientes cabeceras HTTP:

- **X-Content-Type-Options: nosniff**: impide que el navegador intente adivinar el tipo MIME de un recurso, previniendo ataques de *sniffing* de contenido.
- **X-Frame-Options: DENY**: impide que la página pueda ser cargada dentro de un elemento `<iframe>`, previniendo ataques de tipo *Clickjacking*.
- **X-XSS-Protection: 1; mode=block**: activa el filtro XSS integrado del navegador. Aunque se trata de una cabecera considerada legada, sigue siendo útil como capa adicional de protección.
- **Content-Security-Policy**: es la cabecera de seguridad más robusta, ya que define explícitamente desde qué orígenes se permite cargar scripts, estilos, imágenes y otros recursos.

Validación de datos y prevención de vulnerabilidades (OWASP Top 10)

Durante el desarrollo del Servlet de registro se implementó la validación de los datos del lado del servidor para los campos de nombre, correo electrónico, contraseña y edad del usuario, evitando depender únicamente de la validación realizada en el navegador. Esta práctica responde directamente a la categoría A04 (Diseño Inseguro) del OWASP Top 10, la cual recomienda validar siempre del lado del servidor.

Asimismo, se consideraron las categorías A02 (Fallas Criptográficas) y A03 (Inyección). Para mitigar A02, las contraseñas se almacenan utilizando `BCrypt.hashpw(clave, BCrypt.gensalt(12))`, evitando el uso de algoritmos obsoletos como MD5 o SHA1, y exigiendo el uso de HTTPS.

Para mitigar A03, las consultas a la base de datos se realizan mediante `PreparedStatement`, evitando la concatenación de cadenas que podría derivar en ataques de inyección SQL; y en las vistas JSP se utiliza la etiqueta `<c:out>`, que escapa automáticamente el contenido HTML, previniendo ataques de tipo Cross-Site Scripting (XSS) [1].

Introducción a Spring Boot 3.x

Spring Boot elimina la necesidad de configuración manual de Tomcat, del archivo `web.xml` y del empaquetado WAR. En su lugar, la aplicación se distribuye como un archivo JAR ejecutable que incluye un servidor Tomcat embebido, permitiendo que el desarrollador se concentre principalmente en la lógica de negocio [2].

A diferencia del modelo Servlet/WAR tradicional, donde la configuración se realiza mediante `web.xml` y anotaciones, y el despliegue se efectúa sobre un servidor Tomcat externo, en Spring Boot la configuración se centraliza en el archivo `application.properties`, el servidor Tomcat se encuentra embebido en el JAR, las vistas pueden generarse con Thymeleaf, FreeMarker o directamente como JSON para APIs REST, el acceso a datos se simplifica mediante Spring Data JPA y la seguridad se gestiona a través de Spring Security con una configuración mínima.

Además, Spring Boot introduce el principio de Inversión de Control (IoC) e Inyección de Dependencias (DI): en lugar de que cada clase cree sus propias dependencias mediante `new`, el contenedor IoC de Spring las crea y las inyecta automáticamente, gestionando el ciclo de vida de los *beans*. Entre las anotaciones más utilizadas para definir beans se encuentran `@Component` (bean genérico), `@Service` (lógica de negocio), `@Repository` (acceso a datos), `@RestController` (controlador HTTP que retorna JSON) y `@Configuration` (clase de configuración de beans). Todos los componentes anotados de esta manera son detectados automáticamente mediante el mecanismo de *component scanning* de Spring Boot.

Principios SOLID aplicados al proyecto

Los cinco principios SOLID fueron considerados durante el diseño del proyecto, aplicándose de la siguiente manera [3]:

- **Responsabilidad Única (S):** cada clase tiene una única razón para cambiar. Por ejemplo, `UsuarioDAO` se encarga exclusivamente del acceso a la base de datos, `RegistroServlet`

únicamente procesa las solicitudes HTTP, y PasswordUtil se limita al manejo del hashing de contraseñas.

- **Abierto/Cerrado (O):** el sistema permanece abierto a la extensión pero cerrado a la modificación. Por ejemplo, sería posible agregar un EmailNotificacionFilter sin necesidad de modificar el código existente de RegistroServlet.
- **Sustitución de Liskov (L):** una subclase debe poder reemplazar a su superclase sin alterar el comportamiento del programa. En este proyecto, implementaciones como MySQLUsuarioDAO y PostgreSQLUsuarioDAO podrían implementar la misma interfaz UsuarioDAO.
- **Segregación de Interfaces (I):** las interfaces deben ser pequeñas y específicas. Esto permitiría, por ejemplo, separar UsuarioLectorDAO (solo lectura) de UsuarioEscritorDAO (solo escritura).
- **Inversión de Dependencias (D):** las clases deben depender de abstracciones y no de implementaciones concretas. En este proyecto, el Servlet depende de la interfaz UsuarioDAO, y no directamente de MySQLUsuarioDAO.

Reglas de Clean Code aplicadas

Durante el desarrollo se aplicaron las principales reglas de Clean Code propuestas por Robert C. Martin: el uso de nombres significativos para métodos y variables (por ejemplo, buscarUsuarioPorEmail() en lugar de nombres poco descriptivos); funciones pequeñas que realicen una sola tarea, evitando métodos con más de 20 líneas; código autodocumentado que evite la necesidad de comentarios para explicar lógica confusa; la eliminación de números mágicos, declarando constantes como `private static final int BCRYPT_COSTO = 12;`; el manejo explícito de excepciones, capturando excepciones específicas como SQLException o IOException en lugar de Exception genérica; y el principio DRY (*Don't Repeat Yourself*), extrayendo la lógica de validación repetida hacia una clase común como ValidadorUtil.

GitFlow aplicado al trabajo en equipo

Para la organización del trabajo colaborativo del equipo se aplicó el flujo de ramas propuesto por GitFlow [4]. La rama main contiene la versión estable y desplegable del proyecto, y únicamente recibe *merges* provenientes de ramas release/*. La rama develop constituye

la rama de integración continua del equipo y sirve como base para la creación de ramas feature/*. Cada nueva funcionalidad se desarrolla en una rama feature/nombre, creada a partir de develop y nombrada de acuerdo con la funcionalidad correspondiente, como por ejemplo feature/registro-usuario. Finalmente, las correcciones urgentes en producción se realizan mediante ramas hotfix/nombre, creadas a partir de main y posteriormente fusionadas tanto en main como en develop.

Tabla comparativa: Java/JSP, PHP y ASP.NET

A continuación, se presenta una tabla comparativa entre las principales tecnologías de desarrollo web del lado del servidor, considerando ocho criterios relevantes.

Criterio	Java (JSP/Servlets)	PHP	ASP.NET
Tipado	Fuertemente tipado (estático)	Débilmente tipado (dinámico)	Fuertemente tipado (estático)
Servidor	Apache Tomcat, Jetty, WildFly	Apache, Nginx con PHP-FPM	IIS
Curva de aprendizaje	Media-alta	Baja	Media
Rendimiento	Alto (compilado a bytecode)	Medio (interpretado)	Alto (compilado a CLR)
Portabilidad	Alta (multiplataforma)	Alta	Limitada (principalmente Windows, aunque .NET Core mejora esto)
Ecosistema	Amplio (Spring, Jakarta EE)	Amplio (Laravel, Symfony)	Amplio (.NET, Entity Framework)
Seguridad	Robusta, con JSTL anti-XSS y PreparedStatement	Requiere disciplina manual del desarrollador	Robusta, con Razor y validación integrada
Uso en la	industria	Sistemas	empresariales, banca, gobierno

Sitios web, CMS, e- commerc e	Aplicacion es empresaria les Microsoft
---	--

RESULTADOS OBTENIDOS

Se desarrolló correctamente una aplicación web basada en JSP y Servlets que permite registrar usuarios mediante un formulario web. Durante las pruebas realizadas se verificó el correcto funcionamiento de las validaciones implementadas para el nombre, correo electrónico, contraseña y edad del usuario. La aplicación mostró mensajes de error cuando los datos ingresados no cumplían los requisitos establecidos, y permitió completar el registro cuando toda la información era válida.

Asimismo, se realizaron pruebas de seguridad para verificar la efectividad de las medidas implementadas. En las pruebas de Cross-Site Scripting (XSS), se comparó el comportamiento de una versión insegura, en la cual el script ingresado era ejecutado por el navegador, frente a una versión segura, en la que el uso de la etiqueta `<c:out>` de JSTL escapaba automáticamente el contenido, mostrándolo como texto plano. De igual forma, se realizó una prueba de inyección SQL, comprobando que el uso de PreparedStatement impide la ejecución de comandos maliciosos inyectados a través de los campos del formulario.

Finalmente, los datos fueron procesados por el Servlet y presentados correctamente en la página de éxito mediante la directiva `forward()`, demostrando la comunicación adecuada entre las vistas JSP y la lógica del servidor.

CONCLUSIONES

Se logró implementar satisfactoriamente una aplicación web utilizando JSP y Servlets, comprendiendo el flujo de procesamiento de solicitudes HTTP entre cliente y servidor, así como el ciclo de vida completo de un Servlet.

El uso de Apache Tomcat permitió desplegar y ejecutar la aplicación correctamente, verificando la integración entre el servidor web y los componentes desarrollados en Java.

La práctica fortaleció conocimientos sobre validación de formularios del lado del servidor, manejo de dependencias con Maven, aplicación de principios de seguridad alineados con OWASP Top 10, y desarrollo de aplicaciones web dinámicas utilizando tecnologías Java empresariales.

Adicionalmente, la aplicación de los principios SOLID y las reglas de Clean Code permitió organizar el código del proyecto de forma modular y mantenible, mientras que el uso de GitFlow facilitó el trabajo colaborativo y ordenado dentro del equipo.

Referencias

- [1] K. Ahmad y M. Karim, “A Method to Prevent SQL Injection Attack using an Improved Parameterized Stored Procedure,” *International Journal of Advanced Computer Science and Applications*, vol. 12, n.º 6, 2021. [10.14569/IJACSA.2021.0120636](https://doi.org/10.14569/IJACSA.2021.0120636).
- [2] I. Shabani, E. Mëziu, B. Berisha y T. Biba, “Design of Modern Distributed Systems based on Microservices Architecture,” *International Journal of Advanced Computer Science and Applications*, vol. 12, n.º 2, 2021. [10.14569/IJACSA.2021.0120220](https://doi.org/10.14569/IJACSA.2021.0120220).
- [3] M. G. Saeed, M. T. Alasaady, F. L. Malallah y K. H. Faraj, “Three Levels Quality Analysis Tool for Object Oriented Programming,” *International Journal of Advanced Computer Science and Applications*, vol. 9, n.º 11, 2018. [10.14569/IJACSA.2018.091173](https://doi.org/10.14569/IJACSA.2018.091173).
- [4] J. L. C. Valdez, H. F. Medina, G. A. V. Contreras y A. H. G. González, “Collaborative Integrated Model in Agile Software Development (MDSIC/MDSIC-M) - Case Study and Practical Advice,” *International Journal of Advanced Computer Science and Applications*, vol. 10, n.º 7, 2019. [10.14569/IJACSA.2019.0100759](https://doi.org/10.14569/IJACSA.2019.0100759).

ANEXOS

Repositorio: <https://github.com/Zaida-tm18/PracticaJSP-UTEQ.git>

Creación de interfaz con Java

localhost:8080/practicaJSP_war_exploded/registro

Crear cuenta

Nombre completo *

Correo electronico *

Edad *

Contraseña (min. 8 caracteres) *

Confirmar contraseña *

Crear cuenta

¿Ya tienes cuenta? [Inicia sesion](#)

Código de la interfaz con Java

```
package ec.edu.uteq.servlet;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.mindrot.jbcrypt.BCrypt;
import java.io.IOException;

@WebServlet(urlPatterns = {"/registro"})
public class RegistroServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
```

```

        req.getRequestDispatcher("/WEB-INF/views/registro.jsp")
            .forward(req, resp);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        req.setCharacterEncoding("UTF-8");

        String nombre = req.getParameter("nombre");
        String email = req.getParameter("email");
        String clave = req.getParameter("clave");
        String clave2 = req.getParameter("clave2");
        String edadStr = req.getParameter("edad");

        StringBuilder errores = new StringBuilder();

        if (nombre == null || nombre.trim().isEmpty()) {
            errores.append("El nombre es obligatorio. ");
        }
        if (email == null || !email.matches("^[A-Za-z0-9+_.\\-]+@[A-Za-z0-9\\.\\-]+$.+")) {
            errores.append("El email no es valido. ");
        }
        if (clave == null || clave.length() < 8) {
            errores.append("La clave debe tener min. 8 caracteres. ");
        }
        if (clave != null && !clave.equals(clave2)) {
            errores.append("Las claves no coinciden. ");
        }

        int edad = 0;
        try {
            edad = Integer.parseInt(edadStr);
            if (edad < 1 || edad > 120) {
                errores.append("La edad debe estar entre 1 y 120. ");
            }
        } catch (NumberFormatException e) {
            errores.append("La edad debe ser un numero entero. ");
        }

        if (errores.length() > 0) {
            req.setAttribute("errores", errores.toString());
            req.setAttribute("nombre", nombre);
            req.setAttribute("email", email);
            req.setAttribute("edad", edadStr);
            req.getRequestDispatcher("/WEB-INF/views/registro.jsp")
                .forward(req, resp);
            return;
        }

        String hashClave = BCrypt.hashpw(clave, BCrypt.gensalt(12));

        resp.sendRedirect(req.getContextPath() + "/login?registrado=true");
    }
}

```

Interfaz de Login con Java

localhost:8080/practica/SP_war_exploded/login?registrado=true

🔍 ☆ ⚙

Iniciar sesion

¡Cuenta creada exitosamente! Ahora puedes iniciar sesion.

Correo electronico
ztaipem@uteq.edu.ec

Contraseña
.....

Entrar

¿No tienes cuenta? [Registrate](#)

Registro de servlet

